Endsem - Analysis and Design of Algorithms (CSE222)

Name: Section: Roll No. :

1. Consider the following flow network $G = \{V, E\}$ with flow and capacity mentioned on their edges. For every edge $e \in E$, the following image shows $f(e)/c(e)$ where $f(e)$ is the flow and $c(e)$ is the capacity on e respectively.

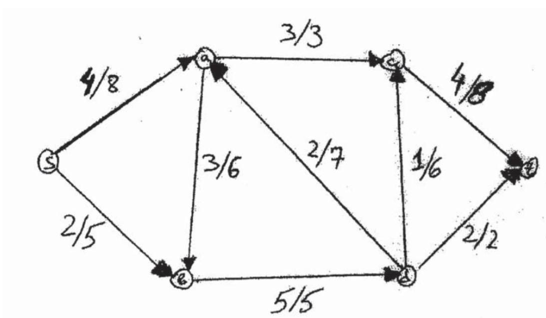
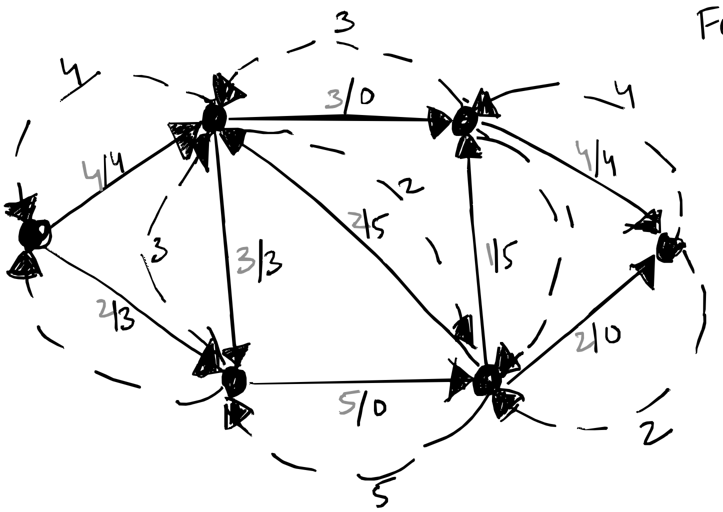


Figure 1: Flow Network

(a) (3 points) Compute the residual graph G_f for the above flow function f.



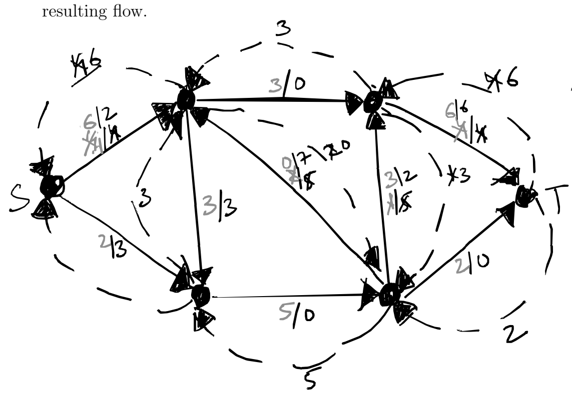
For TA:- For every forward (or thick) edge e, we have $f(e)/b(e)$ where $f(e)$ is its flow & $b(e) = c(e) - f(e)$

- Do not deduct marks if an edge e is not drawn for $b(e) = 0$
- If $f(e) > 0$, draw $e' = (b,a)$ where $e = (a,b)$ & set $c(e') = f(e)$.

Rubric:- If backward edges (e.g. dashed) not drawn then deduct 1.

- If capacity on backward edges incorrect then deduct 1.
- If $b(e)$ on forward edges incorrect then deduct 1.

(b) (3 points) Perform *one* step of Ford-Fulkerson's algorithm on your above computed G_f and draw the resulting flow.



For TA: - Do not deduct marks if backward edges not drawn.
- Total flow out of s = total flow in T. If above not true then deduct 1.
- There can be multiple solutions.
- Check for every vertex (excet s & t) flow in = flow out. If not deduct 1.
- For every edge e, $f(e) \leq c(e)$, else deduct 1.
- For every edge both $f(e)/b(e)$ or $f(e)/c(e)$ are fine.

2. Consider an integer-capacitated directed graph $G = (V, E)$ and a *maximum flow* f given to you. Now suppose the capacity of a specific edge e is increased by 1. Design an $O(|V| + |E|)$-time algorithm to compute the maximum flow in the modified network.

For TA:
Run FF award 0.5
Run 1 step FF give 1.
Run 1 step FF on residual graph give 2.
Run 1 step FF on residual of modified n/w give 3

(a) (3 points) Algorithm description (Precisely one/two English sentences are sufficient)

- Compute the residual graph $\tilde{G}_f$ of the modified network & the flow function f'.
- Run one step of FF on $\tilde{G}_f$ & return flow function.

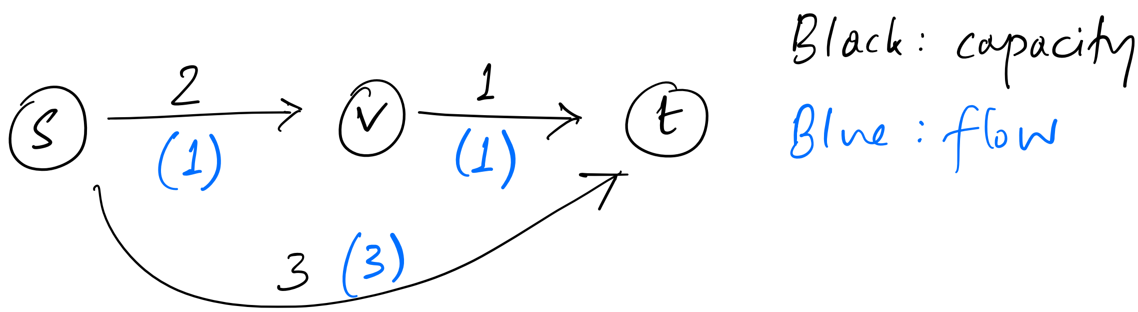
(b) (3 points) Justify the correctness of your algorithm in two-three sentences precisely.

- Let G_f is the residual graph of G for f.
- Since f is maximum flow of G, hence there is no s-t path in G_f.
- Now, if there is a s-t path in the residual graph $\tilde{G}_f$ of the of the modified network, then this path can only carry 1 more unit of flow. (Deduct 1 if any form of this fact is missing)
- Since G is integer-capacitated directed graph & every intermediate flow are also integer, hence at most one more ff step will return the maximum flow. [Deduct 1 if any form of this fact is missing]

3. State True or False. As usual, if True, write a small justification. If false, show a counter-example.

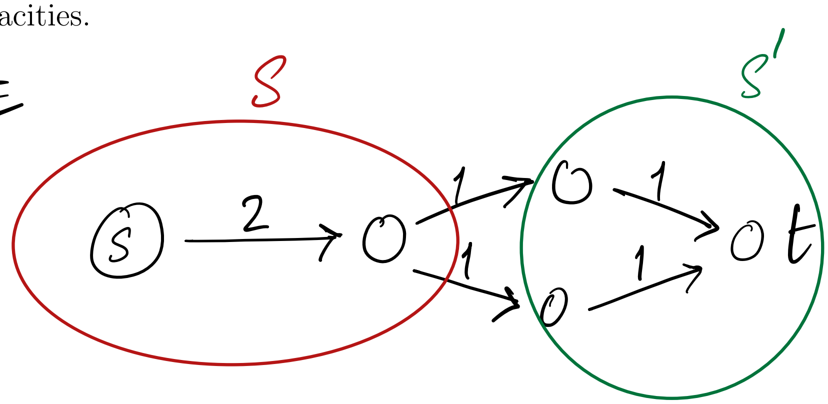
(a) (2 points) Let G be an arbitrary flow network, with a source s, a sink t, and a positive integer capacity c_e on every edge e. If f is a maximum $s-t$ flow in G, then f saturates every edge out of s with flow (i.e., for all edges e out of s, we have $f(e) = c(e)$).

FALSE



(b) (2 points) Let G be an arbitrary flow network, with a source s, a sink t, and a positive integer capacity c_e on every edge e; and let (A, B) be a minimum $s-t$ cut with respect to these capacities $\{c_e : e \in E\}$. Now suppose we add 1 to every capacity; then (A, B) is still a minimum $s-t$ cut with respect to these new capacities.

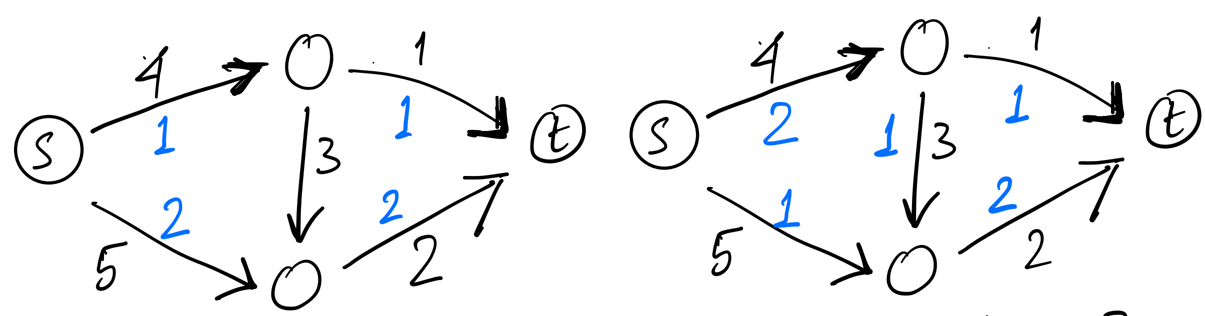
FALSE



(S, S') is a min-cut in G, but not a min-cut after increasing capacities.

(c) (2 points) Consider a flow network G with n nodes and m edges such that the capacity on each edge is unique. Then the max flow is also unique.

False:



Clearly, both are max-flows with value 3 (since min-cut has value 3) but they are distinct flows.

[Remark: No score without justification]

4. You are supposed to evaluate n endsem scripts for ADA. However, the latest season of Squid Games just dropped and of course you need to watch them right away. You get in touch with r many of your friends, each of whom owes you a favor and ask them to help you with the task. Friend j only promises to evaluate $0 < b_j \leq n$ many scripts. Further, you also need to make sure that any of your friends does not evaluate the script of somebody who resides in the same hostel as them.

(a) (4 points) Design an algorithm that runs in time polynomial in n and r by showing a reduction to max-flow with precisely defined vertices, edges, and capacities.

Construction: We create a bipartite graph $G = (X \cup J, E)$

$0.25 + 0.25$ { X: vertices corresponding to each endsem script
J: " " " " friend.

0.5 { E : $\exists$ edge $(x f)$ in E if and only if the endsem script x can be evaluated by friend j (this information is known since you know who lives in which hostel)

Add source s and an edge $(\vec{sx})$ $\forall x \in X$ } $0.25 + 0.25$
" sink t " " " $(\vec{jt})$ $\forall j \in J$

Capacities: $c(sx) = 1$ $\forall x \in X$, $c(xj) = 1$ (or $+\infty$) $\forall (xj) \in E$, } $0.5 + 0.5 + 0.5$
$c(jt) = b_j$ $\forall j \in J$

Find an s-t max-flow: let the value be λ^*. } 1
Then you need to evaluate $n - \lambda^*$ scripts.
[Full credit even for correct construction + just saying max s-t flow

(b) (3 points) Briefly justify why finding a max-flow solves your problem and its running time.

Correctness: We claim that a feasible s-t flow of value λ $\Leftrightarrow$ a feasible evaluation of λ scripts by your friends.

$\Rightarrow$ Consider any integer s-t flow of value λ. Any path carrying positive flow will have an edge of the form (xj), $x \in X$, $j \in J$. Take the union of all such edges incident on $j \in J$. Assign those scripts to friend j. This assignment is feasible since (1) $(xj) \in E$ iff j can evaluate script x & (2) flow out of j cannot exceed b_j which means at most b_j flow is coming into j $\Rightarrow$ at most b_j scripts assigned to j. (+1)

Rough work

$\Leftarrow$ Suppose $\exists$ a valid assignment of scripts to friends such that they evaluate λ scripts in all. Assign flow values as follows on the basis of that:

(a) $f(sx) = 1$ if ~~scr~~ script x is evaluated by someone

(b) $f(xj) = 1$ if " " " " by friend j

(c) $f(jt) = \#$ scripts assigned to friend j.

It is clear as daylight that f is a feasible flow and since $f(sx) = 1$ $\forall x$ s.t. x is evaluated, value of flow $= \lambda$. (+1)

Runtime: Number of edges in the graph $= n + r + n \cdot r$.
The max-flow is $\min(n, \sum_{j \in J} b_j)$.

Hence an upper bound is $O(n^2 r)$. (+1)

[Remark]: Any reasonable and precise justification will be awarded either fully or partly.